ВІННИЦЬКИЙ НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ

ІНСТИТУТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ ТА КОМП'ЮТЕРНОЇ ІНЖЕНЕРІЇ

КАФЕДРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Дослідження та реалізація базових динамічних структур даних

Звіт про виконання лабораторної роботи № 3

Виконав (ла): ст. 2 курсу, групи 2ПІ-25Б

Коваль Олександр

Перевірив: викладач Карась О.В.

Вінниця 2026

Тема: Дослідження та реалізація базових структур даних. Двозв'язний список із розгалуженнями.

Мета: набути навичок роботи з динамічними структурами та покажчиками у мові C++; реалізувати двозв'язний список із гілками для збереження й обробки числових послідовностей.

ЗАВДАННЯ

Розробити програму на мові C++, яка зберігає дані виду «Ім'я | Послідовність чисел» та виконує наступні операції:
1. Додає елементи до двозв'язного списку;
2. Знаходить і роздруковує елемент за його ім'ям;
3. Видаляє елемент зі списку;
4. Підраховує кількість елементів у списку;
5. Роздруковує весь список.
Послідовність містить від 1 до N чисел. Для довжини понад K організувати гілку (K < N).

КОД ПРОГРАМИ

```
#include <iostream>
#include <string>

const int K = 3;
const int N = 10;

struct BranchNode {
   int value;
   BranchNode* next = nullptr;
};

struct Node {
   std::string name;
   int numbers[K]{};
   int count = 0;
   BranchNode* branch = nullptr;
   Node* prev = nullptr;
   Node* next = nullptr;
};

struct List {
   Node* head = nullptr;
```

```
  Node* tail = nullptr;
};

bool add(List& list, const std::string& name, const int numbers[], int count) {
  bool has_name = false;
  for (int i = 0; i < int(name.length()); i++) {
    if (name[i] != ' ' && name[i] != '\t' && name[i] != '\r')
      has_name = true;
  }
  if (!has_name || count < 1 || count > N || numbers == nullptr)
    return false;

  Node* node = new Node;
  node->name = name;
  node->count = count;
  if (count > K) node->count = K;
  for (int i = 0; i < node->count; i++)
    node->numbers[i] = numbers[i];

  BranchNode* branch_tail = nullptr;
  for (int i = K; i < count; i++) {
    BranchNode* number = new BranchNode;
    number->value = numbers[i];
    if (node->branch == nullptr) node->branch = number;
    else branch_tail->next = number;
    branch_tail = number;
  }

  node->prev = list.tail;
  if (list.tail == nullptr) list.head = node;
  else list.tail->next = node;
  list.tail = node;
  return true;
}

Node* find(const List& list, const std::string& name) {
  Node* node = list.head;
  while (node != nullptr) {
    if (node->name == name) return node;
    node = node->next;
  }
  return nullptr;
}

void destroy_node(Node* node) {
```

```
  while (node->branch != nullptr) {
    BranchNode* old = node->branch;
    node->branch = old->next;
    delete old;
  }
  delete node;
}

bool remove(List& list, const std::string& name) {
  Node* node = find(list, name);
  if (node == nullptr) return false;

  if (node->prev == nullptr) list.head = node->next;
  else node->prev->next = node->next;
  if (node->next == nullptr) list.tail = node->prev;
  else node->next->prev = node->prev;
  destroy_node(node);
  return true;
}

int size(const List& list) {
  int count = 0;
  Node* node = list.head;
  while (node != nullptr) {
    count++;
    node = node->next;
  }
  return count;
}

void print_node(const Node* node) {
  if (node == nullptr) {
    std::cout << "Елемент не знайдено.\n";
    return;
  }
  std::cout << node->name << " | Основні числа:";
  for (int i = 0; i < node->count; i++)
    std::cout << ' ' << node->numbers[i];
  std::cout << " | Гілка:";
  if (node->branch == nullptr) std::cout << " відсутня";
  BranchNode* number = node->branch;
  while (number != nullptr) {
    std::cout << ' ' << number->value;
    number = number->next;
  }
```

```
    std::cout << '\n';
}

void print_list(const List& list) {
    if (list.head == nullptr) std::cout << "Список порожній.\n";
    Node* node = list.head;
    while (node != nullptr) {
        print_node(node);
        node = node->next;
    }
}

void clear(List& list) {
    while (list.head != nullptr) {
        Node* old = list.head;
        list.head = old->next;
        destroy_node(old);
    }
    list.tail = nullptr;
}

bool read_numbers(int numbers[], int count) {
    for (int i = 0; i < count; i++) {
        while (std::cin.peek() == ' ' || std::cin.peek() == '\t' ||
               std::cin.peek() == '\r')
            std::cin.get();
        if (std::cin.peek() == '\n' || std::cin.eof()) {
            std::cin.get();
            return false;
        }
        if (!(std::cin >> numbers[i])) {
            std::cin.clear();
            std::string rest;
            std::getline(std::cin, rest);
            return false;
        }
    }
    while (std::cin.peek() == ' ' || std::cin.peek() == '\t' ||
           std::cin.peek() == '\r')
        std::cin.get();
    if (std::cin.eof()) return true;
    if (std::cin.peek() == '\n') {
        std::cin.get();
        return true;
    }
```

```
  std::string rest;
  std::getline(std::cin, rest);
  return false;
}

int main() {
  List list;
  int choice = -1;
  std::cout  << "Двозв'язний список із розгалуженнями. K = 3, N = 10\n";
  while (choice != 0) {
    std::cout << "\n1. Додати\n2. Знайти\n3. Видалити\n"
          << "4. Кількість\n5. Друк списку\n6. Приклад\n0. Вийти\nВибір: ";
    if (!read_numbers(&choice, 1)) {
      if (std::cin.eof()) break;
      choice = -1;
      std::cout << "Введіть ціле число від 0 до 6.\n";
      continue;
    }
    switch (choice) {
      case 0:
        break;
      case 1: {
        std::string name;
        int count, numbers[N];
        std::cout << "Ім'я: ";
        if (!std::getline(std::cin, name)) break;
        std::cout << "Кількість чисел: ";
        if (!read_numbers(&count, 1) || count < 1 || count > N) {
          std::cout << "Некоректна кількість чисел.\n";
          break;
        }
        std::cout << "Числа одним рядком: ";
        if (!read_numbers(numbers, count)) {
          std::cout << "Некоректна послідовність чисел.\n";
          break;
        }
        if (add(list, name, numbers, count))
          std::cout << "Елемент додано.\n";
        else std::cout << "Ім'я не може бути порожнім.\n";
        break;
      }
      case 2:
      case 3: {
        std::string name;
        std::cout << "Ім'я: ";
```

```
            if (!std::getline(std::cin, name)) break;
            if (choice == 2) print_node(find(list, name));
            else if (remove(list, name)) std::cout << "Елемент видалено.\n";
            else std::cout << "Елемент не знайдено.\n";
            break;
        }
        case 4:
            std::cout << "Кількість елементів: " << size(list) << '\n';
            break;
        case 5:
            print_list(list);
            break;
        case 6: {
            int a[] = {5, 12};
            int b[] = {10, 20, 30, 40, 50, 60};
            add(list, "Запис_1", a, 2);
            add(list, "Запис_2", b, 6);
            std::cout << "Додано 2 приклади.\n";
            break;
        }
        default:
            std::cout << "Введіть ціле число від 0 до 6.\n";
      }
   }
   clear(list);
   std::cout << "Кінець\n";
   return 0;
}
```

РЕЗУЛЬТАТИ ТЕСТУВАННЯ ТА РОБОТИ ПРОГРАМИ

1. Результати виконання програми зі зразковими даними. Послідовність вибору: 6, 5, 2 (Запис_2), 3 (Запис_1), 4, 5, 0.

```
Двозв'язний список із розгалуженнями. K = 3, N = 10

1. Додати
2. Знайти
3. Видалити
4. Кількість
5. Друк списку
6. Приклад
0. Вийти
Вибір: Додано 2 приклади.

1. Додати
2. Знайти
3. Видалити
4. Кількість
5. Друк списку
6. Приклад
0. Вийти
Вибір: Запис_1 | Основні числа: 5 12 | Гілка: відсутня
Запис_2 | Основні числа: 10 20 30 | Гілка: 40 50 60

1. Додати
2. Знайти
3. Видалити
4. Кількість
5. Друк списку
6. Приклад
0. Вийти
Вибір: Ім'я: Запис_2 | Основні числа: 10 20 30 | Гілка: 40 50 60

1. Додати
2. Знайти
3. Видалити
4. Кількість
5. Друк списку
6. Приклад
0. Вийти
Вибір: Ім'я: Елемент видалено.

1. Додати
2. Знайти
3. Видалити
4. Кількість
5. Друк списку
6. Приклад
0. Вийти
Вибір: Кількість елементів: 1

1. Додати
2. Знайти
3. Видалити
4. Кількість
5. Друк списку
6. Приклад
```

```
0. Вийти
Вибір: Запис_2 | Основні числа: 10 20 30 | Гілка: 40 50 60

1. Додати
2. Знайти
3. Видалити
4. Кількість
5. Друк списку
6. Приклад
0. Вийти
```

БЛОК-СХЕМА АЛГОРИТМУ (ГОСТ)

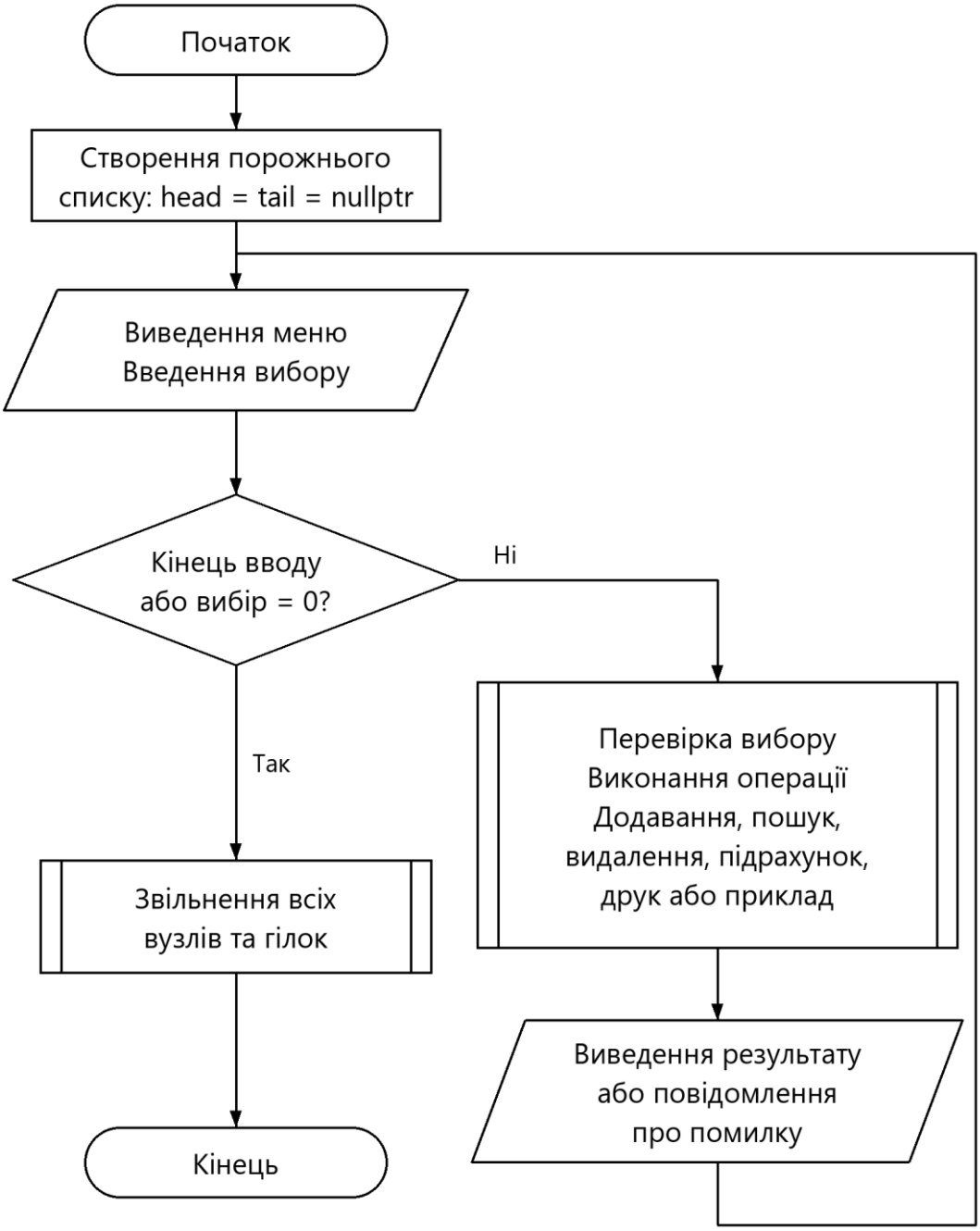


Рисунок 1 — Основний цикл програми

Вінниця 2026

БЛОК-СХЕМА ДОДАВАННЯ ЕЛЕМЕНТА (ГОСТ)

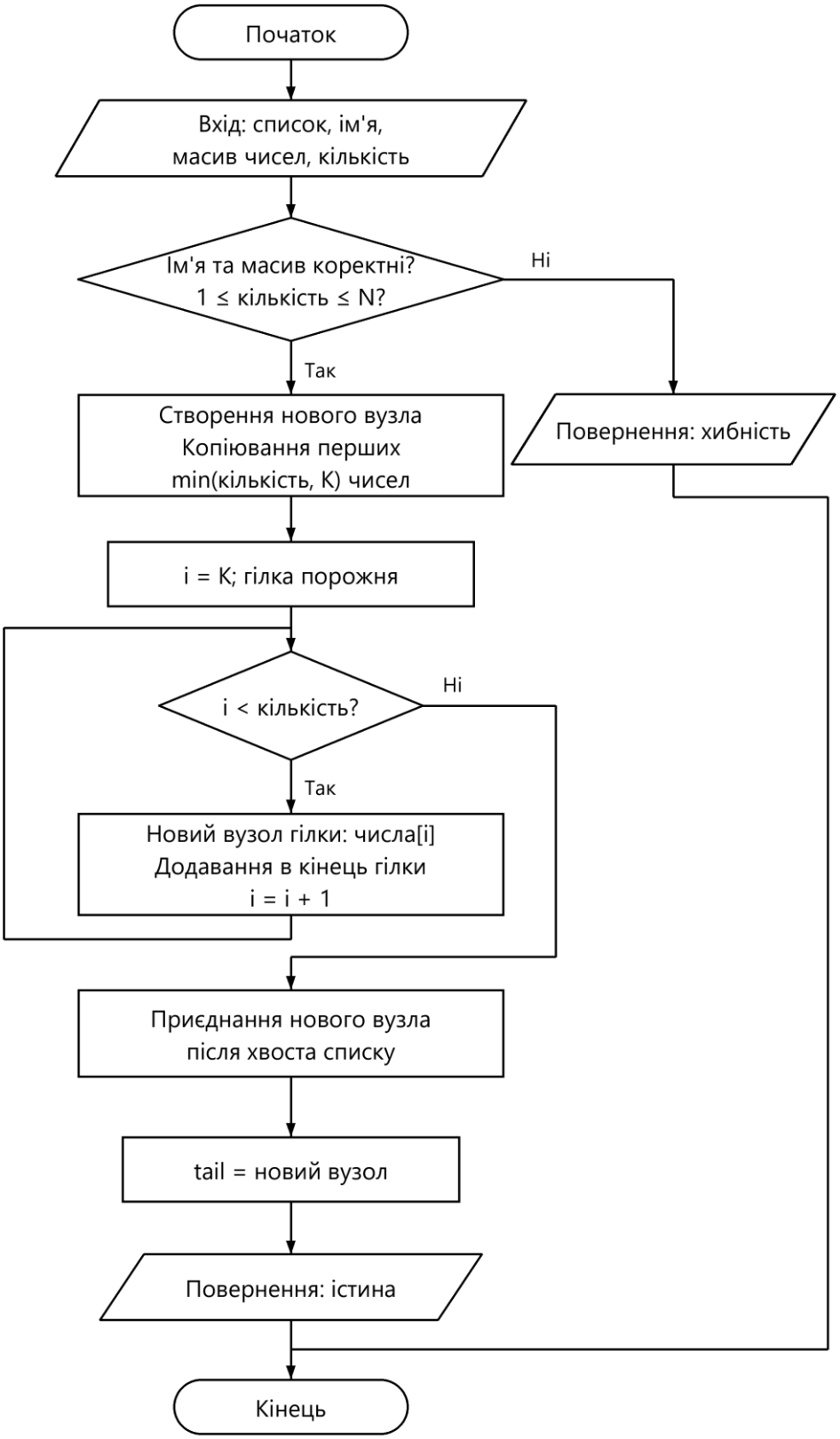


Рисунок 2 — Додавання запису та формування гілки

Вінниця 2026

Висновок: У результаті виконання лабораторної роботи реалізовано двозв'язний список із розгалуженнями. Програма дозволяє додавати записи, знаходити їх за ім'ям, видаляти, підраховувати та роздруковувати список. Перші K = 3 числа зберігаються в основному вузлі, а решта послідовно додається до окремої гілки. Реалізація використовує структури, масиви, покажчики, цикли та звичайні функції; підключено лише бібліотеки iostream і string. Перевірка вводу запобігає додаванню некоректних записів. При видаленні та завершенні роботи звільняється пам'ять основних вузлів і гілок. Усі вісім автоматичних перевірок пройдено. Підрахунок виконується обходом списку й займає O(m) часу для m записів.

Вінниця 2026